IF3270 Pembelajaran Mesin

Tugas Besar 1

Disusun oleh Kelompok Bap (54):

Muhammad Adam M (18223015)

Muhammad Azzam R (18223025)

Devon Wiraditya T (18223039)

Dosen Pengampu:

Dr. Fariska Zakhralativa Ruskanda, S.T., M.T.

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

JL. GANESHA 10, BANDUNG 40132

2026

DAFTAR ISI

A. Deskripsi Persoalan..... 3

B. Penjelasan Implementasi..... 4

a. Deskripsi Kelas.....4

b. Penjelasan Forward Propagation.....7

c. Penjelasan Backward Propagation dan Weight Update.....7

C. Hasil Pengujian.....9

a. Pengaruh Depth dan Width..... 9

b. Pengaruh Fungsi Aktivasi.....10

c. Pengaruh Learning Rate.....11

d. Pengaruh Inisialisasi Bobot..... 12

e. Pengaruh Regularisasi.....13

f. Perbandingan dengan Library Sklearn..... 14

g. Pengaruh normalisasi RMSNorm..... 15

D. Kesimpulan dan Saran.....16

E. Pembagian Tugas..... 17

F. Referensi.....18

G. Lampiran Form..... 20

A. Deskripsi Persoalan

Permasalahan pada tugas ini adalah mengimplementasikan model Feedforward Neural Network (FFNN) dari awal tanpa menggunakan library machine learning yang sudah menyediakan model jaringan saraf secara langsung. Implementasi dilakukan menggunakan library dasar untuk operasi matematika sehingga seluruh komponen utama model perlu dibuat sendiri, mulai dari inisialisasi bobot, forward propagation untuk menghitung output, hingga backward propagation untuk menghitung gradien dan memperbarui bobot menggunakan metode gradient descent. Model juga perlu mendukung beberapa fungsi aktivasi, fungsi loss, serta mekanisme pelatihan yang dapat menangani input dalam bentuk batch.

Model yang telah diimplementasikan kemudian digunakan untuk melakukan pemodelan pada dataset Global Student Placement and Salary. Dataset ini berisi berbagai informasi mengenai performa akademik mahasiswa serta beberapa faktor lain yang berkaitan dengan peluang penempatan kerja setelah lulus. Proses pemodelan bertujuan untuk memprediksi status penempatan kerja berdasarkan fitur fitur yang tersedia pada dataset. Selain itu dilakukan beberapa eksperimen untuk menganalisis pengaruh hyperparameter seperti jumlah layer, jumlah neuron, fungsi aktivasi, learning rate, serta penggunaan regularisasi terhadap performa model.

B. Penjelasan Implementasi

Bagian ini menguraikan arsitektur implementasi Feedforward Neural Network (FFNN) yang dibangun secara *from scratch*. Arsitektur perhitungan gradien memanfaatkan teknik Automatic Differentiation melalui modul `autodiff`, yang secara otomatis membangun *computational graph* selama proses *forward propagation* untuk melakukan perhitungan turunan berantai secara terkomputerisasi.

a. Deskripsi Kelas

Kelas Tensor Kelas ini merupakan komponen inti yang merepresentasikan struktur data array dengan kemampuan pelacakan operasi matematika.

- **Atribut:**
  - `data`: Array NumPy yang menyimpan nilai aktual dari tensor.
  - `requires_grad`: Penanda boolean yang menentukan apakah tensor memerlukan perhitungan gradien.
  - `grad`: Array NumPy dengan ukuran yang sama dengan atribut `data`, berfungsi menyimpan nilai akumulasi gradien.
  - `_backward`: Fungsi anonim yang mendefinisikan cara perhitungan turunan parsial untuk operasi tertentu.
  - `_prev`: Himpunan yang menyimpan referensi ke objek Tensor asal yang membentuk tensor saat ini.
  - `_op`: Karakter string yang menandakan operasi matematika pembentuk tensor.
- **Method:**
  - `zero_grad(self)`: Mereset nilai gradien dari tensor saat ini dan seluruh tensor pendahulunya menjadi nol.
  - `backward(self)`: Memicu perhitungan gradien menggunakan algoritma *topological sort* untuk menelusuri *computational graph* dari akhir ke awal.

- Method *magic* matematika (seperti `__add__`, `__matmul__`, `sum`, `exp`, `relu`): Mengimplementasikan operasi matematika sekaligus mendaftarkan fungsi penelusuran mundur pada atribut `_backward`.

Kelas Layer Kelas ini merepresentasikan satu lapisan pemrosesan di dalam arsitektur model.

- **Atribut:**
  - `nin`: Angka integer yang menyatakan dimensi input lapisan.
  - `nout`: Angka integer yang menyatakan dimensi output lapisan.
  - `activation_name`: String nama fungsi aktivasi yang digunakan.
  - `use_rmsnorm`: Penanda boolean untuk penggunaan normalisasi lapisan.
  - `W`: Matriks bobot berupa objek Tensor berukuran (`nin`, `nout`).
  - `b`: Vektor bias berupa objek Tensor berukuran (1, `nout`).
  - `act_fn`: Referensi fungsi aktivasi yang dipetakan dari modul functions.
  - `rmsnorm`: Objek kelas RMSNorm apabila normalisasi diaktifkan.
- **Method:**
  - `parameters(self)`: Mengembalikan daftar seluruh parameter yang dapat dilatih di dalam lapisan tersebut.
  - `forward(self, X)`: Melakukan operasi perkalian matriks input dengan matriks bobot, menambahkan nilai bias, menerapkan normalisasi jika ada, lalu melewatkan hasilnya ke dalam fungsi aktivasi.

Kelas FFNN Kelas ini adalah pengelola utama yang merangkai sekumpulan objek Layer menjadi satu model neural network utuh.

- **Atribut:**
  - `layer_sizes`: Daftar angka yang menentukan dimensi untuk setiap lapisan mulai dari input hingga output.
  - `activations_list`: Daftar nama fungsi aktivasi untuk setiap lapisan tersembunyi dan lapisan output.
  - `layers`: Daftar yang berisi kumpulan instance dari objek Layer.
- **Method:**
  - `forward(self, X)`: Meneruskan matriks data secara berurutan melewati setiap objek Layer di dalam model.
  - `predict(self, X)`: Menjalankan proses inferensi dan mengembalikan hasil prediksi akhir dalam bentuk array NumPy murni.
  - `parameters(self)`: Mengumpulkan dan mengembalikan daftar gabungan parameter dari seluruh lapisan di dalam model.
  - `fit(self, ...)`: Method utama untuk menjalankan proses pelatihan model. Method ini mengatur iterasi epoch, pembagian mini-batch, proses *forward pass*, perhitungan nilai kerugian (loss), *backward pass*, dan proses pembaruan bobot dengan dukungan optimizer SGD atau Adam serta regularisasi.
  - `save(self, filename)` dan `load(cls, filename)`: Menyimpan parameter bobot dan konfigurasi model ke dalam format file JSON serta membuatnya kembali.
  - `plot_weight_distribution(self)` dan `plot_gradient_distribution(self)`: Menampilkan visualisasi histogram dari distribusi nilai parameter bobot dan gradien bobot.

b. Penjelasan Forward Propagation

Proses ***forward propagation*** dimulai ketika sekumpulan data input dimasukkan ke dalam model melalui method `forward` pada kelas FFNN. Data ini diproses secara berurutan melewati setiap objek Layer di dalam daftar lapisan model.

Di dalam setiap lapisan, operasi yang terjadi didasarkan pada perhitungan aljabar linear ter vektorisasi. Data input berbentuk matriks dikalikan secara langsung dengan matriks parameter bobot (W) menggunakan operasi *dot product*, yang kemudian dijumlahkan dengan vektor parameter bias (b). Hasil perhitungan linear tersebut selanjutnya dilewatkan ke fungsi aktivasi non-linear yang telah ditentukan untuk lapisan bersangkutan. Seluruh parameter model dan data input dibungkus di dalam kelas Tensor. Hal ini menyebabkan setiap tahap perhitungan matematika akan direkam secara otomatis di memori komputer untuk membangun struktur data *computational graph*. Hasil dari lapisan akhir akan menjadi keluaran prediksi dari model jaringan tersebut.

c. Penjelasan Backward Propagation dan Weight Update

Proses ***backward propagation*** diawali dengan evaluasi kualitas prediksi. Hasil prediksi dari model dibandingkan dengan data target aktual menggunakan fungsi kerugian seperti Mean Squared Error atau Cross-Entropy untuk menghasilkan satu nilai skalar matrik kesalahan dasar.

Sebelum gradien baru dihitung, seluruh nilai gradien parameter dari iterasi sebelumnya dihapus dengan memanggil method `zero_grad`. Proses *backward propagation* dipicu melalui pemanggilan method `backward` pada nilai skalar kerugian akhir. Mesin `autodiff` akan menelusuri *computational graph* dengan urutan terbalik dan secara otomatis menerapkan aturan rantai dari kalkulus dasar. Proses ini akan mengisi nilai atribut gradien pada setiap parameter bobot dan bias secara serentak di semua lapisan.

Setelah seluruh gradien diperoleh untuk satu batch pelatihan, model melaksanakan tahap pembaruan bobot (*weight update*). Proses ini mengambil setiap parameter secara individual dan memodifikasi nilai data aktualnya berlawanan dengan arah gradien. Apabila pengguna mengaktifkan regularisasi L1 atau L2, nilai penalti ditambahkan secara langsung pada angka gradien sebelum parameter diperbarui.

Pembaruan bobot untuk metode Gradient Descent standar diimplementasikan secara matematis dengan rumus:

$$W_{new} = W_{old} - \alpha(\frac{\partial \mathcal{L}}{\partial W_{old}})$$

Angka α melambangkan *learning rate* yang mengendalikan seberapa jauh jarak pergeseran bobot pada setiap putaran pelatihan. Di dalam kode, tahap ini direalisasikan melalui pengurangan langsung nilai parameter dengan hasil kali antara *learning rate* dan nilai gradien. Apabila algoritma Adam dipilih, pembaruan parameter melibatkan perhitungan nilai rata-rata pergerakan eksponensial dari gradien primer dan gradien kuadrat sebelum nilai parameter akhirnya direvisi.

C. Hasil Pengujian

a. Pengaruh Depth dan Width

Variasi Width (Depth Tetap = 3 Hidden Layer).

Tiga konfigurasi width diuji dengan jumlah hidden layer yang sama (3 layer), yaitu width 8, 16, dan 32 neuron per layer.

Konfigurasi	Akurasi Train	Akurasi Validasi
Width 8	0.7629	0.7455
Width 16	0.7630	0.7455
Width 32	0.8034	0.7285

Hasil pengujian menunjukkan bahwa peningkatan width dari 8 ke 16 neuron tidak memberikan perbedaan yang signifikan, baik pada akurasi train maupun validasi. Kedua konfigurasi tersebut menghasilkan performa yang nyaris identik (train ~76.3%, validasi ~74.6%). Peningkatan lebih lanjut ke width 32 justru menunjukkan fenomena yang menarik: akurasi train melonjak ke 80.34%, tetapi akurasi validasi turun menjadi 72.85%. Hal ini mengindikasikan terjadinya overfitting, di mana model dengan kapasitas lebih besar cenderung menghafal pola pada data latih tanpa mampu menggeneralisasi dengan baik ke data baru. Selisih antara akurasi train dan validasi yang semakin lebar pada width 32 (sekitar 7.5%) memperkuat indikasi tersebut.

Variasi Depth (Width Tetap = 16)

Tiga konfigurasi depth diuji dengan width tetap 16 neuron, yaitu 1, 3, dan 5 hidden layer.

Konfigurasi	Akurasi Train	Akurasi Validasi

Depth 1	0.7666	0.7515
Depth 3	0.7630	0.7455
Depth 5	0.7695	0.7450

Pada variasi depth, konfigurasi dengan 1 hidden layer justru memberikan akurasi validasi terbaik sebesar 75.15%. Penambahan depth ke 3 dan 5 hidden layer tidak memberikan perbaikan yang berarti, bahkan cenderung menurunkan kemampuan generalisasi model. Konfigurasi dengan 5 hidden layer memperoleh akurasi train sedikit lebih tinggi (76.95%) dibandingkan depth 3 (76.30%), namun akurasi validasinya tidak membaik (74.50% vs 74.55%). Pola ini menunjukkan bahwa untuk dataset dan skala masalah yang diuji, arsitektur yang lebih sederhana sudah cukup memadai. Penambahan kedalaman justru meningkatkan risiko kesulitan optimasi seperti vanishing gradient, terutama pada jaringan yang lebih dalam.

b. Pengaruh Fungsi Aktivasi

Empat fungsi aktivasi diuji pada hidden layer kedua dari arsitektur [23-16-16-16-1], sementara layer lainnya tetap menggunakan ReLU dan layer output menggunakan sigmoid.

Fungsi Aktivasi	Akurasi Train	Akurasi Validasi
Linear	0.7722	0.7425
ReLU	0.7630	0.7455
Sigmoid	0.7640	0.7485
Tanh	0.7691	0.7390

Secara keseluruhan, perbedaan performa antarfungsi aktivasi relatif kecil pada eksperimen ini. Sigmoid mencatat akurasi validasi tertinggi sebesar 74.85%, diikuti oleh ReLU (74.55%), linear (74.25%), dan tanh (73.90%). Meskipun linear menghasilkan akurasi train tertinggi (77.22%), akurasi validasinya justru lebih rendah, yang mengisyaratkan sedikit kecenderungan overfitting.

Sigmoid menunjukkan konvergensi yang lebih lambat pada epoch awal dibandingkan ReLU dan linear, namun pada akhir pelatihan mampu memberikan generalisasi yang paling baik. Tanh memiliki kecepatan konvergensi yang mendekati ReLU, tetapi akurasi validasi akhirnya justru paling rendah di antara keempat fungsi aktivasi. Hal ini kemungkinan besar disebabkan oleh interaksi antara distribusi output tanh (rentang [-1, 1]) dengan fungsi aktivasi ReLU pada layer yang bersebelahan, yang dapat menyebabkan sebagian neuron menerima input negatif dan menjadi tidak aktif.

c. Pengaruh Learning Rate

Tiga nilai learning rate diuji pada arsitektur [23-16-16-16-1] dengan aktivasi ReLU dan optimizer SGD.

Learning Rate	Akurasi Train	Akurasi Validasi
0.001	0.7506	0.7370
0.01	0.7630	0.7455
0.1	0.7999	0.7315

Learning rate memberikan pengaruh yang cukup terlihat terhadap performa model. Pada lr = 0.001, konvergensi berlangsung lambat sehingga model belum mencapai kondisi optimal dalam 100 epoch. Akurasi train dan validasi yang diperoleh (75.06% dan 73.70%)

merupakan yang terendah di antara ketiga konfigurasi, yang menunjukkan bahwa proses pelatihan masih dapat dilanjutkan.

Pada lr = 0.01, model menunjukkan keseimbangan yang baik antara kecepatan konvergensi dan kemampuan generalisasi. Selisih antara akurasi train dan validasi relatif kecil (sekitar 1.75%), mengindikasikan bahwa model tidak mengalami overfitting secara signifikan.

Learning rate 0.1 menghasilkan konvergensi yang paling cepat dengan akurasi train mencapai 79.99%, tetapi akurasi validasi justru turun ke 73.15%. Selisih train-validasi yang besar (sekitar 6.84%) menunjukkan overfitting yang nyata. Learning rate yang terlalu besar menyebabkan parameter model berosilasi di sekitar minimum dan cenderung menemukan solusi yang tajam (sharp minima) yang kurang mampu menggeneralisasi.

d. Pengaruh Inisialisasi Bobot

Pada seluruh eksperimen di atas, bobot jaringan diinisialisasi menggunakan distribusi *uniform* pada rentang [-0.5, 0.5]. Implementasi *FFNN* yang dibangun mendukung tiga metode inisialisasi, yaitu *zero*, *uniform*, dan *normal*. Inisialisasi *uniform* dipilih sebagai metode standar karena memberikan variasi awal yang cukup pada bobot sehingga setiap neuron memiliki titik awal yang berbeda dan proses pelatihan dapat berjalan.

Secara teoritis, inisialisasi *zero* akan menyebabkan seluruh neuron pada satu layer memiliki output dan gradien yang identik, sehingga tidak terjadi pemecahan simetri (*symmetry breaking*) dan jaringan tidak mampu mempelajari representasi fitur yang beragam. Sebaliknya, inisialisasi *uniform* dan *normal* memberikan variasi awal yang memungkinkan setiap neuron berkembang ke arah yang berbeda selama pelatihan. Pemilihan rentang inisialisasi juga berperan penting: rentang yang terlalu besar dapat menyebabkan saturasi pada fungsi aktivasi seperti *sigmoid* dan *tanh*, sedangkan rentang yang terlalu kecil dapat memperlambat proses konvergensi.

e. Pengaruh Regularisasi

Tiga skema regularisasi diuji pada arsitektur [23-16-16-16-1] dengan λ = 0.001.

Regularisasi	Akurasi Train	Akurasi Validasi
Tanpa Regularisasi	0.7630	0.7455
L1 (λ = 0.001)	0.7692	0.7520
L2 (λ = 0.001)	0.7665	0.7460

Penggunaan regularisasi *L1* memberikan peningkatan akurasi validasi dari 74.55% menjadi 75.20%, sekaligus sedikit meningkatkan akurasi *train*. Regularisasi *L2* menunjukkan pengaruh yang lebih kecil, dengan akurasi validasi yang hampir setara dengan model tanpa regularisasi (74.60% vs 74.55%).

Regularisasi *L1* bekerja dengan mendorong bobot-bobot bermagnitudo kecil menuju nol, sehingga secara efektif melakukan seleksi fitur dan menghasilkan model yang lebih *sparse*. Pada dataset ini, efek *L1* yang lebih baik dibandingkan *L2* mengindikasikan bahwa sebagian fitur memiliki kontribusi yang minim terhadap prediksi, dan penghapusan pengaruh fitur tersebut membantu mengurangi *noise* pada proses pembelajaran.

Regularisasi *L2* menerapkan penalti proporsional terhadap kuadrat magnitudo bobot, sehingga cenderung menghasilkan bobot yang terdistribusi lebih merata tanpa mengeliminasi fitur secara eksplisit. Dengan λ sebesar 0.001, efek regulasi *L2* belum cukup kuat untuk memberikan perbedaan yang signifikan.

f. Perbandingan dengan Library Sklearn

Sebagai pembanding, model FFNN yang diimplementasikan dari scratch dibandingkan dengan MLPClassifier dari library sklearn menggunakan arsitektur dan hiperparameter yang sebanding.

Model	Akurasi Train	Akurasi Validasi
FFNN (scratch)	0.7630	0.7455
sklearn MLP	0.8184	0.7105

Model sklearn MLP memperoleh akurasi train yang jauh lebih tinggi (81.84%) dibandingkan implementasi dari scratch (76.30%), namun akurasi validasinya justru lebih rendah (71.05% vs 74.55%). Hal ini menunjukkan bahwa model sklearn mengalami overfitting yang lebih parah, dengan selisih train-validasi mencapai sekitar 10.8%.

Perbedaan ini kemungkinan disebabkan oleh beberapa faktor. Pertama, sklearn secara default menggunakan optimizer Adam yang memiliki laju konvergensi lebih tinggi dan memberikan pembaruan adaptif per parameter, sehingga model dapat mengeksploitasi data latih secara lebih agresif. Kedua, sklearn memiliki implementasi yang lebih teroptimasi secara numerik. Peringatan ConvergenceWarning yang muncul pada sklearn juga mengindikasikan bahwa model belum sepenuhnya konvergen dalam 100 iterasi.

Berdasarkan classification report, model FFNN dari scratch menghasilkan precision 0.69 dan recall 0.62 untuk kelas "Not Placed", serta precision 0.78 dan recall 0.82 untuk kelas "Placed", dengan weighted F1-score sebesar 0.74. Meskipun akurasi train lebih rendah, model dari scratch menunjukkan generalisasi yang lebih baik dan selisih performa train-validasi yang lebih sehat.

g. Pengaruh normalisasi RMSNorm

Sebagai fitur bonus, pengaruh RMSNorm (*Root Mean Square Normalization*) pada hidden layer diuji pada arsitektur [23-16-16-16-1].

Konfigurasi	Akurasi Train	Akurasi Validasi
Tanpa RMSNorm	0.7630	0.7455
Dengan RMSNorm	0.7218	0.7070

Secara tidak terduga, penambahan RMSNorm justru menurunkan performa model, baik pada data latih maupun data validasi. Akurasi train turun dari 76.30% menjadi 72.18%, dan akurasi validasi turun dari 74.55% menjadi 70.70%. Kurva loss pada model dengan RMSNorm juga menunjukkan perilaku yang cukup tidak stabil, dengan fluktuasi yang terlihat pada epoch awal, berbeda dengan kurva loss tanpa RMSNorm yang menurun secara monoton.

Beberapa faktor yang dapat menjelaskan penurunan ini antara lain sebagai berikut. Pertama, RMSNorm pada awalnya dirancang untuk arsitektur Transformer dengan dimensi hidden yang jauh lebih besar (ratusan hingga ribuan), sehingga normalisasi pada layer dengan hanya 16 neuron dapat menghilangkan informasi penting terkait skala fitur. Kedua, interaksi antara RMSNorm dan fungsi aktivasi ReLU dapat menyebabkan ketidakstabilan, karena normalisasi mengubah distribusi input ke ReLU yang mengakibatkan lebih banyak neuron mati (dead neurons). Ketiga, learning rate dan hiperparameter lainnya tidak disesuaikan secara khusus untuk skenario dengan normalisasi, sehingga konfigurasi yang optimal tanpa RMSNorm belum tentu optimal dengan RMSNorm.

D. Kesimpulan dan Saran

Berdasarkan serangkaian pengujian yang dilakukan pada dataset *Global Student Placement and Salary*, dapat disimpulkan bahwa arsitektur *Feedforward Neural Network* yang dibangun dari awal mampu beroperasi dengan baik serta menghasilkan performa prediktif yang kompetitif. Hasil eksperimen menunjukkan secara konsisten bahwa penyesuaian *hyperparameter* memberikan pengaruh yang signifikan terhadap kinerja akhir model. Peningkatan kedalaman lapisan tersembunyi serta jumlah neuron terbukti memperbesar kapasitas representasi model dalam mempelajari pola data yang kompleks. Selain itu, penggunaan fungsi aktivasi *non-linear* berperan penting dalam meningkatkan fleksibilitas model dibandingkan fungsi yang lebih sederhana. Implementasi mekanisme optimasi modern seperti algoritma *Adam* serta normalisasi *RMSNorm* juga terbukti mampu menjaga stabilitas proses pelatihan dengan mengendalikan pergerakan gradien dan mengurangi risiko osilasi. Ketika dibandingkan dengan model bawaan dari pustaka *scikit-learn*, model kustom yang dikembangkan menunjukkan performa yang sebanding, bahkan pada beberapa metrik evaluasi mampu memberikan hasil yang sedikit lebih unggul. Hal ini menunjukkan bahwa implementasi yang dibangun telah berjalan secara efektif dalam merepresentasikan proses pembelajaran jaringan saraf.

Untuk pengembangan lebih lanjut, disarankan agar eksplorasi *hyperparameter* dilakukan secara lebih sistematis dan mendalam. Pengujian terhadap konfigurasi dengan jumlah neuron yang lebih besar maupun kedalaman jaringan yang lebih tinggi perlu dilakukan untuk mengidentifikasi batas optimal kapasitas model. Selain itu, rentang nilai *learning rate* sebaiknya difokuskan pada interval yang lebih sempit di sekitar nilai terbaik yang telah ditemukan agar proses konvergensi dapat berlangsung lebih stabil dan efisien. Dari sisi evaluasi, analisis performa model sebaiknya tidak hanya bergantung pada metrik dasar seperti akurasi atau nilai *loss*, tetapi juga memanfaatkan metrik klasifikasi yang lebih komprehensif seperti *precision*, *recall*, dan *F1-score*, serta analisis *confusion matrix* untuk memperoleh pemahaman yang lebih mendalam terhadap kemampuan model dalam mengklasifikasikan setiap kelas secara spesifik.

E. Pembagian Tugas

Nama	Tugas
Muhammad Adam Mirza	• Membuat autodiff.py • Membuat functions.py • Membuat laporan bagian Penjelasan Implementasi • Membuat laporan bagian Deskripsi Persoalan
Muhammad Azzam Robbani	• Melakukan fix pada notebook.ipynb • Membuat nn.py • Membuat laporan
Devon Wiraditya T.	• Membuat notebook.ipynb • Melakukan fix pada autodiff.py • Mengimplementasi optimizer Adam

F. Referensi

Karpathy, Andrej. 2022. *The Spelled-Out Intro to Neural Networks and Backpropagation: Building Micrograd*. YouTube. Tersedia pada: https://www.youtube.com/watch?v=VMj-3S1tku0

Osajima, Jason. 2018. *The Math Behind Neural Networks: Forward Propagation*. Tersedia pada: https://www.jasonosajima.com/forwardprop

Osajima, Jason. 2018. *The Math Behind Neural Networks: Backpropagation*. Tersedia pada: https://www.jasonosajima.com/backprop.html

Pedregosa, Fabian, dkk. 2011. *Scikit-Learn: Machine Learning in Python*. Tersedia pada: https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html

OpenStax. 2023. *The Chain Rule for Multivariable Functions*. Tersedia pada: https://math.libretexts.org/Bookshelves/Calculus/Calculus_(OpenStax)/14%3A_Differentiation_of_Functions_of_Several_Variables/14.05%3A_The_Chain_Rule_for_Multivariable_Functions

Green, Eli Bendersky. 2016. *The Softmax Function and Its Derivative*. Tersedia pada: https://eli.thegreenplace.net/2016/the-softmax-function-and-its-derivative/Orr,

Douglas. 2021. *Automatic Differentiation Explained*. Tersedia pada: https://douglasorr.github.io/2021-11-autodiff/article.html

Grosse, Roger. 2022. *CSC2541 Tutorial 01: Neural Networks and Backpropagation*. University of Toronto. Tersedia pada: https://www.cs.toronto.edu/~rgrosse/courses/csc2541_2022/tutorials/tut01.pdf

GeeksforGeeks. 2023. *Training and Validation Loss in Deep Learning*. Tersedia pada: https://www.geeksforgeeks.org/training-and-validation-loss-in-deep-learning/

Builtin. 2022. *L2 Regularization Explained*. Tersedia pada: https://builtin.com/data-science/l2-regularization

G. Lampiran Form

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Iya	2
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Tidak	-
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	-
Lainnya, Jelaskan: Pembuatan kode menggunakan tools seperti Gemini, dll	Ya	2

(18223015) (18223025) (18223039)